LESSON PLAN · PYTHON (GR 6–8) INTERMEDIATE: PYTHON GAME CODERS · LEVEL 2

Lesson 8: Capstone: Battle Arena

Module 2 | 60 minutes | game design, functions, data | Grades 7–11 · ages 12–16

Objective: Design, build and playtest a turn-based text battle game with fighter stats in dictionaries, functions for attack and heal, random damage, a game loop and win or lose endings.

Success criteria

- I can store game stats in dictionaries and a list.
- I can write functions that return damage and healing within limits.
- I can build a game loop with win and lose endings.
- I can playtest my game and balance its numbers.

Key vocabulary

- **HP**: Hit points: a fighter's health. At 0, they are defeated.
- **turn-based**: Players take turns, one action at a time.
- **game loop**: The loop that repeats each turn until the game ends.
- **max() / min()**: Give the bigger or smaller of two numbers; used to keep values in limits.
- **balancing**: Changing numbers so a game is fair and fun.
- **playtest**: Playing your own game to find bugs and boring parts.

You will need

- Projector with Kitabash (lesson video, Python Lab)
- Devices: 1 per learner, with a keyboard
- Printed worksheet (optional)
- Printed project brief
- Project files: battle_fighters.py

Timed lesson plan (60 min)

Time	Stage	What happens
5 min	**Warm-up game: Paper Battle**	Play Paper Battle in pairs with a dice. Ask what repeated every turn and when it stopped: that is the game loop.
5 min	**Hook, video and design**	Show the hook and play the capstone video. Thumbs poll: "Who wants a dragon in their arena?" Learners fill in the design sheet: hero, 3 enemies, moves.
8 min	**Live demo and build-along**	Run the live demo, then everyone builds fighters and one attack. Thumbs check: "Did your goblin lose HP?"
5 min	**Helpers, predict and order**	Attack and heal challenge, the last-hit prediction and the battle-turn order. Quick competition: first correct order in the chat wins.

18 min	**Build the Battle Arena**	Learners build the full arena in breakout rooms. Halfway check-in: one full turn works. Teacher visits each room and asks “What happens when HP hits 0?”
6 min	**Debug race**	Race to fix “the goblin that would not fall”. The winner explains why the loop never ended.
9 min	**Playtest swap and show-and-tell**	Pairs playtest each other’s arena three times and suggest one balance change. Back together, two learners run a battle live while the class cheers every hit.
4 min	**Submit and exit ticket**	Learners submit the project with a reflection. Exit ticket: apply_damage(6, 9).

Unplugged warm-up

Paper Battle (5 min game): in pairs, one learner is the hero (100 HP, 3 potions) and the other is a Goblin (45 HP). Take turns rolling a real or online dice: hero damage is dice + 8, goblin damage is dice + 3, a potion heals 25. Keep HP on paper and never go below 0. After two minutes, ask: what repeated every turn, and when did the battle stop? That is the game loop.

Teaching the idea

The capstone combines the whole course. Fighter stats live in dictionaries and enemies in a list, so random.choice can pick one. Small functions do one job and return a number: roll_damage, apply_damage (max keeps HP at 0 or above) and heal_hp (min keeps HP at max or below). A while loop runs the battle while both fighters have HP, and an if after the loop decides the ending. Give time for design on paper first and for playtesting and balancing at the end.

Analogy: The game is like a cricket match with an umpire. The dictionaries are the scoreboard, the functions are the umpire’s rules (out, four, six), and the game loop is the overs being bowled until the match is decided.

Live demo: 1) Show the design sheet: hero stats, three enemies, moves. 2) Type apply_damage and heal_hp and test them with (10, 15) and (90, 20, 100). 3) Type the player and enemy dictionaries and random.choice(enemies). 4) Code one turn without a loop: show stats, ask move, attack, enemy hits back. Run it twice. 5) Wrap it in while player["hp"] > 0 and enemy["hp"] > 0: and add the ending. 6) Play a full battle live, then change the enemy damage to show balancing.

Questions to ask

Ask	Listen for
Why store stats in a dictionary?	All of a fighter’s values stay together under clear names like hp and potions.
What does max(0, hp - damage) protect against?	HP going below zero.
What is the loop condition for the battle?	Both fighters have HP above 0.

Why does the enemy only hit back if its HP is above 0?	A defeated enemy cannot attack.
What is balancing and how do you do it?	Changing numbers like HP and damage until the game is fair; you playtest and adjust.

Differentiation

Support
Provide the build-along file and the helper functions finished. A first goal is one enemy and attack-only; add heal and the enemy list after it works.

Stretch
Add a Player class with attack and heal methods, a special move that can miss (random chance), or a tournament of three enemies in a row with a for loop.

Common misconceptions

Misconception	How to fix it
enemy_hp - damage changes enemy_hp	You must store the result: enemy_hp = enemy_hp - damage.
The loop condition should use or	Use and: the battle continues only while BOTH have HP.
Random games cannot be tested	Test the helper functions with fixed numbers and playtest many times.
A function cannot change a dictionary passed to it	Changing a key inside the function changes the same dictionary outside.

Assessment

Project rubric (also on the project brief):

Criterion	1 · Beginning	2 · Developing	3 · Secure	4 · Excellent
Program works	Crashes or never ends	One turn works but the battle does not end properly	Full battles end with the right winner every time	Works perfectly and handles odd input and empty potions
Data and functions	No dictionaries or functions	Some stats in variables; one function	Stats in dictionaries, enemy list and helper functions that return values	Clean functions (or a class) with no repeated code and clear names
Game design and balance	No choices for the player	Attack only; the game is always won or always lost	Attack and heal with random damage; the game feels fair	Extra moves or enemies, and balance changes explained from playtests

Testing and presentation	Never tested; no comments	Tested once; few comments	Helpers tested and code commented; clear messages each turn	Playtested by others, improved, and presented with a short design explanation

Exit ticket
Your hero has 6 HP and the goblin hits for 9. What should apply_damage(6, 9) return, and why?

Homework
Play your Battle Arena with two people at home. Ask what would make it more exciting, add one new move or enemy, and write down your balance changes.

Answer key — Kitabash activities

1. Python Lab: attack and heal (perform · python lab)

Goals checked: Attack takes away HP · Heal never goes over the maximum · Uses functions with dictionaries

One solution:

```
def attack(attacker, defender, damage):
    defender["hp"] = max(0, defender["hp"] - damage)
    return defender["hp"]

def heal(fighter, amount):
    fighter["hp"] = min(fighter["max_hp"], fighter["hp"] + amount)
    return fighter["hp"]

hero = {"name": "Hero", "hp": 90, "max_hp": 100}
goblin = {"name": "Goblin", "hp": 30, "max_hp": 30}
print(attack(hero, goblin, 40))
print(heal(hero, 20))
```

2. Predict the output: the last hit (perform · code output)

3. Plan a battle turn (perform · ordering)

Order: Show both fighters' stats → Ask the player for a move → Do the player's move (attack or heal) → If the enemy still has HP, it hits back → Add 1 to the turn counter → Loop again while both have HP left

4. Project: Battle Arena (perform · python lab)

Goals checked: apply_damage stops at 0 · heal_hp stops at the maximum · A battle of attacks ends · Unknown moves are handled · Uses random, functions and a game loop

Example program:

```
# Battle Arena
# Made by: Meera
import random

def roll_damage(low, high):
    return random.randint(low, high)

def apply_damage(hp, damage):
    return max(0, hp - damage)

def heal_hp(hp, amount, max_hp):
    return min(max_hp, hp + amount)

def show_stats(player, enemy):
    print(f"{player['name']}: {player['hp']}/{player['max_hp']} HP, potions:
{player['potions']}")
    print(f"{enemy['name']}: {enemy['hp']} HP")

enemies = [
    {"name": "Goblin", "hp": 45, "low": 4, "high": 9},
    {"name": "Stone Golem", "hp": 60, "low": 3, "high": 7},
    {"name": "Fire Bat", "hp": 35, "low": 6, "high": 11},
]

print("=== BATTLE ARENA ===")
player = {"name": input("Name your hero: ").strip(), "hp": 100, "max_hp": 100,
"potions": 3}
enemy = random.choice(enemies)
print(f"A wild {enemy['name']} appears!")

turn = 1
while player["hp"] > 0 and enemy["hp"] > 0:
    print(f"--- Turn {turn} ---")
    show_stats(player, enemy)
    move = input("attack or heal? ").strip().lower()
    if move == "attack":
        damage = roll_damage(8, 15)
        enemy["hp"] = apply_damage(enemy["hp"], damage)
        print(f"You strike for {damage} damage!")
    elif move == "heal":
        if player["potions"] > 0:
            player["potions"] -= 1
            player["hp"] = heal_hp(player["hp"], 25, player["max_hp"])
            print("You drink a potion. Glug glug!")
        else:
            print("No potions left!")
    else:
        print("Unknown move! You trip over your own feet.")

    if enemy["hp"] > 0:
        hit = roll_damage(enemy["low"], enemy["high"])
```

```
        player["hp"] = apply_damage(player["hp"], hit)
        print(f"The {enemy['name']} hits you for {hit}!")
    turn += 1

if player["hp"] > 0:
    print(f"Victory! {player['name']} defeated the {enemy['name']}!")
else:
    print(f"Defeat! The {enemy['name']} was too strong this time.")
print("GAME OVER")
```

5. Debug it: the goblin that would not fall (probe · python lab)

Goals checked: The battle ends · HP ends at exactly 0

One solution:

```
import random

enemy_hp = 30
turns = 0
while enemy_hp > 0:
    damage = random.randint(5, 10)
    enemy_hp = max(0, enemy_hp - damage)
    turns += 1
print(f"The goblin is defeated in {turns} turns!")
```

Answer key — worksheet

1. **Key words:** HP = Hit points: a fighter's health. At 0, they are defeated. · turn-based = Players take turns, one action at a time. · game loop = The loop that repeats each turn until the game ends. · max() / min() = Give the bigger or smaller of two numbers; used to keep values in limits.
2. **Plan your program:** Goals: Attack takes away HP; Heal never goes over the maximum; Uses functions with dictionaries. One solution: def attack(attacker, defender, damage): defender["hp"] = max(0, defender["hp"] - damage) return defender["hp"] def heal(fighter, amount): fighter["hp"] = min(fighter["max_hp"], fighter["hp"] + amount) return fighter["hp"] hero = {"name": "Hero", "hp": 90, "max_hp": 100} goblin = {"name": "Goblin", "hp": 30, "max_hp": 30} print(attack(hero, goblin, 40)) print(heal(hero, 20))
3. **Predict the output: the last hit:** A. 0
4. **Plan a battle turn:** 1 Show both fighters' stats · 2 Ask the player for a move · 3 Do the player's move (attack or heal) · 4 If the enemy still has HP, it hits back · 5 Add 1 to the turn counter · 6 Loop again while both have HP left
5. **Plan your program:** Goals: The battle ends; HP ends at exactly 0. One solution: import random enemy_hp = 30 turns = 0 while enemy_hp > 0: damage = random.randint(5, 10) enemy_hp = max(0, enemy_hp - damage) turns += 1 print(f"The goblin is defeated in {turns} turns!")
6. **My plan:** A clear plan that covers every requirement.
7. **Checklist:** All requirements met.
8. **Two stars and a wish:** Kind, specific feedback and an improvement.

Kitabash original curriculum.